

A Unified and Time-Efficient Multi-Agent Framework for Data Discovery

Yunhao Xiao*
Huazhong Agricultural University
Wuhan, China
Yunhao_Xiao@webmail.hzau.edu.cn

Ying Wang*
Huazhong Agricultural University
Wuhan, China
yingwang@mail.hzau.edu.cn

Michael Bewong
Charles Sturt University
Bathurst, Australia
mbewong@csu.edu.au

Selasi Kwashie
Charles Sturt University
Bathurst, Australia
skwashie@csu.edu.au

Xiaoxia Li†
Huazhong Agricultural University
Wuhan, China
lixiaoxia@mail.hzau.edu.cn

Zaiwen Feng†
Huazhong Agricultural University
Wuhan, China
zaiwen.feng@mail.hzau.edu.cn

Abstract

Discovering related tables for join and union operations on the Web is a critical task hindered by a fundamental trade-off: existing monolithic methods are either fast but semantically shallow, or deep but computationally prohibitive. To resolve this dilemma, we introduce a unified and time-efficient multi-agent framework that reimagines table discovery as a cooperative, goal-driven process. At the core of our solution lies the Three-Layer Cascaded Filtering (TLCF) algorithm, a novel mechanism that synergistically combines rapid metadata filtering, semantic vector search, and deep contextual reasoning from Large Language Models (LLMs) to balance efficiency and accuracy. This powerful algorithm is orchestrated within our framework by six specialized agents that adaptively apply the TLCF's stages to the distinct requirements of both join and union discovery. Comprehensive experiments demonstrate that our framework consistently and significantly outperforms a wide range of state-of-the-art baselines across diverse Web corpora, establishing it as a practical, robust, and scalable solution for time-efficient, high-precision data discovery.

CCS Concepts

• **Information systems** → **Information integration**; • **Computing methodologies** → Multi-agent systems.

Keywords

Data Discovery; Multi-Agent; Semantic Search; Large Language Models

ACM Reference Format:

Yunhao Xiao, Ying Wang, Michael Bewong, Selasi Kwashie, Xiaoxia Li, and Zaiwen Feng. 2026. A Unified and Time-Efficient Multi-Agent Framework for Data Discovery. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792615>

*Both authors contributed equally to this research.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792615>

1 Introduction

The World Wide Web has evolved from a network of documents into the largest, most diverse repository of structured data. This "Web of Data" encompasses billions of HTML tables and millions of datasets published via open data portals [33], forming a global data commons with immense potential for knowledge discovery [15, 22, 24, 28] and large-scale analytics [2, 4, 17]. However, the very nature of the Web—its scale, heterogeneity, and lack of curation—poses a formidable challenge for data discovery [3, 7, 11, 14]. A critical task in harnessing this data is finding *joinable* tables [8, 9, 35] to enrich datasets and *unionable* tables [1, 10, 11, 19, 23] to consolidate information. These operations are foundational for countless applications, yet are hindered by a fundamental trade-off between efficiency and accuracy that existing methods have failed to resolve.

Existing approaches to table discovery are often monolithic, relying on a single strategy that proves inadequate for the multi-faceted nature of Web data. As illustrated by the practical discovery task in Figure 1, these methods falter in distinct ways. Traditional String Matching techniques such as JOSIE [35] are efficient but brittle, failing on simple lexical variations (e.g., "NVIDIA" vs. "NVDA"). Indexing and Search Optimization Methods such as LSH are approximate by design and struggle with the inherent precision-efficiency trade-off, leading to unreliable results. Even more advanced Context and Semantic Enhancement Methods such as D3L [1] can be misled by polysemy, for instance, incorrectly associating the company "Apple" with the fruit. As empirically confirmed by recent benchmarks [7], each of these monolithic strategies fails because it addresses only one facet of the problem, highlighting the need for a more adaptive and comprehensive solution.

To address these limitations, we introduce a unified and time-efficient multi-agent framework that reimagines table discovery as a cooperative, goal-driven process. Our core innovation is to replace the rigid, monolithic pipeline with a flexible society of six specialized agents orchestrated by LangGraph [21]. As depicted in Figure 1, this intelligent division of labor allows our framework to systematically overcome the challenges that hinder prior art. The workflow begins as the ConfiguratorAgent assesses the task's complexity, after which the PlannerAgent and AnalyzerAgent formulate a task-specific strategy. This plan is then executed by the SearcherAgent and MatcherAgent, which employ our novel Three-Layer

Cascaded Filtering (TLCF) algorithm. The TLCF synergistically combines rapid metadata filtering, semantic vector search, and deep contextual reasoning from LLMs to achieve a balance of speed and accuracy. Finally, the AggregatorAgent synthesizes all evidence to produce the final ranked results.

The main contributions of this work are as follows: **A Collaborative Multi-Agent Architecture for Web Table Discovery:** we propose and implement a novel framework employing six specialized agents that transforms table discovery from a monolithic process into an adaptive, collaborative task. This architecture enables intelligent task decomposition and dynamic, query-specific strategy selection. **A Three-Layer Cascaded Filtering (TLCF) Algorithm:** we introduce a hierarchical filtering mechanism that synergistically combines the efficiency of metadata and vector search with the deep contextual reasoning of LLMs. This algorithm is central to resolving the long-standing efficiency-accuracy trade-off in large-scale table discovery. **Comprehensive Experimental Validation and State-of-the-Art Performance:** we conduct an extensive empirical evaluation on three diverse Web data corpora, including a realistically enhanced benchmark with programmatic noise. Our results demonstrate that our framework consistently and significantly outperforms a wide range of established baseline methods across all standard metrics for both join and union discovery tasks, establishing a new state-of-the-art.

2 Related Work

Our research synthesizes advances across three key domains. We first review systems for Web table discovery, then examine pre-trained foundation models for tabular data, and finally discuss the emerging paradigm of LLMs and agent-based systems. This review situates our work as a novel synthesis that leverages an agent-based architecture to resolve core challenges in prior work.

2.1 Web Table Discovery

The discovery of related tables in Web corpora is a long-standing challenge. Early systems like Aurum [12] used heuristics and lightweight embeddings, struggling with deep semantic diversity. More recent, specialized systems have emerged for join and union tasks. For union discovery, Starmie [11] and Santos [19] employ contextual semantics, while for join discovery, methods range from efficient set-overlap techniques like JOSIE [35] to deep learning models like DeepJoin [9].

However, as the comprehensive LakeBench benchmark [7] systematically demonstrated, a fundamental tension between accuracy and efficiency persists; no single, monolithic strategy excels across all discovery scenarios. Our work addresses this gap by proposing an adaptive framework that orchestrates multiple strategies, rather than relying on a single, fixed pipeline.

2.2 Pre-trained Foundation Models for Tabular Data

A parallel line of research focuses on creating universal, task-agnostic table understanding models through large-scale pre-training. A key contribution is TURL [6], which pioneered the pre-training and fine-tuning paradigm for Web tables, learning rich representations effective for various downstream tasks. Following this, other

models [26] have explored different architectures and pre-training objectives. While these foundation models offer impressive semantic capabilities, their high computational cost for pairwise comparison across an entire corpus remains a significant bottleneck. Our TLCF algorithm pragmatically addresses this by leveraging their embeddings in an efficient Layer 2 (L2) vector search layer, reserving full inference for only the most promising candidates.

2.3 LLMs and Agent-based Systems for Data Tasks

The advent of Large Language Models (LLMs) has introduced a new paradigm for data tasks. Recent advancements have significantly enhanced the reasoning capabilities and reliability of these models through techniques such as causal inference and bias mitigation [25, 27, 30, 34]. Building on this foundation, models such as CHORUS [18] and Jellyfish [32] have demonstrated strong zero-shot reasoning for schema matching and other data preparation tasks. However, their direct application to large-scale discovery is hampered by latency and API costs, motivating their use in our framework as a final, high-precision verification step.

To manage complex, multi-step reasoning, agent-based systems have emerged as a powerful architecture for collaborative problem-solving [13, 29]. These systems are now being applied to complex scenarios beyond simple query generation, such as urban planning [16]. Despite these advances, the application of a multi-agent architecture to the large-scale information retrieval problem of Web table discovery remains a nascent area. Our work is the first, to our knowledge, to propose a collaborative multi-agent system specifically designed to orchestrate the synergistic strengths of heuristic filtering, dense retrieval, and LLM-based reasoning for this challenge.

3 Problem Formulation and Proposed Framework

In this section, we first formally define the problem of Web table discovery. We then present our novel multi-agent framework designed to address this challenge, detailing its architecture and the core algorithm it employs.

3.1 Problem Formulation

The primary challenge we address is discovering semantically and structurally related tables within a large, heterogeneous corpus of Web-derived tables. Such a corpus is characterized by a lack of a global schema, inconsistent naming conventions, and diverse data representations.

Formally, given a query table T_q and a large corpus of Web-derived tables $\mathcal{D} = \{T_1, T_2, \dots, T_n\}$, the goal of table discovery is to identify a ranked list of tables $C \subseteq \mathcal{D}$ that are highly relevant to T_q based on a specific integration intent. We focus on the two most common intents: join and union.

Join Discovery Problem. Given a query table T_q , the objective is to find a set of tables $C_{\text{join}} \subseteq \mathcal{D}$ where each table $T_j \in C_{\text{join}}$ is joinable with T_q . This typically requires identifying at least one pair of columns, $c_q \in T_q$ and $c_j \in T_j$, that are semantically and structurally compatible. This can be expressed as finding tables T_j

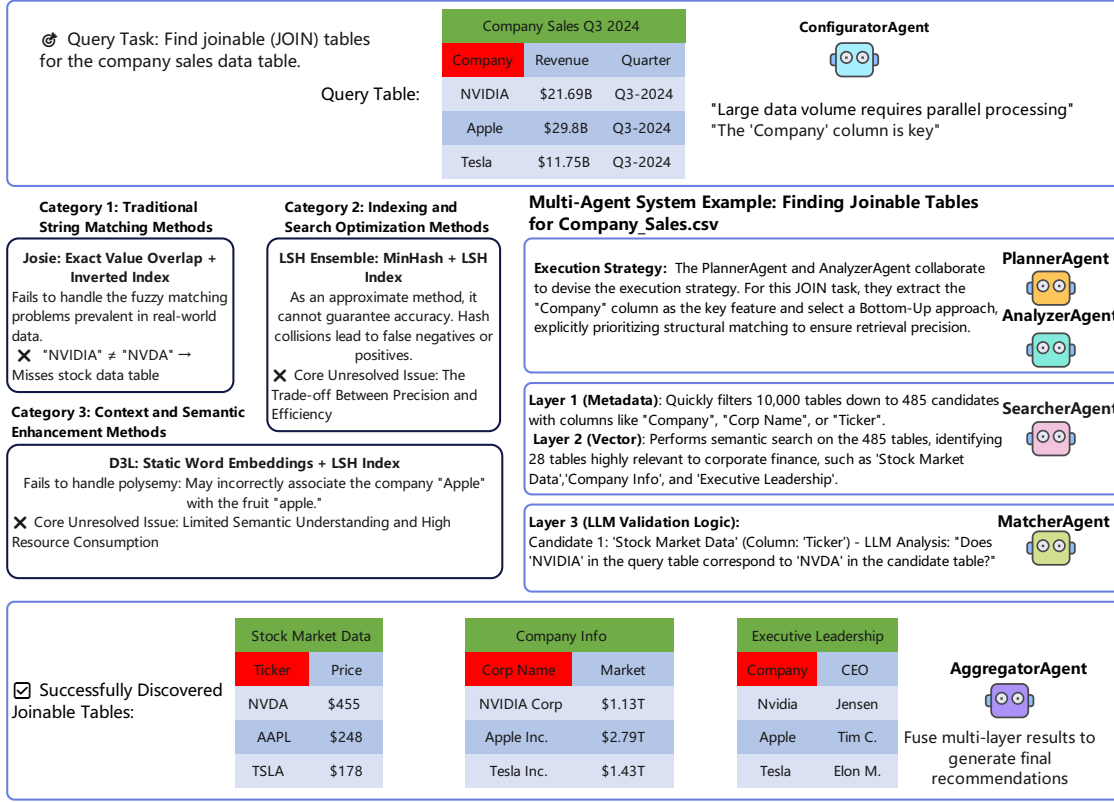


Figure 1: A motivating example for Web table discovery. Given a query sales table, traditional monolithic methods fail due to distinct limitations: (1) String Matching misses lexical variations (e.g., "NVIDIA" vs. "NVDA"); (2) Indexing Methods suffer from the precision-efficiency trade-off; and (3) Early Semantic Methods are misled by polysemy (e.g., company "Apple" vs. fruit "apple"). In contrast, our multi-agent framework employs a collaborative, multi-stage process where specialized agents (Configurator, Planner, Searcher, etc.) work in concert to overcome these challenges and correctly discover a diverse set of joinable tables.

such that:

$$\exists(c_q \in T_q, c_j \in T_j) : \text{similarity}(c_q, c_j) > \theta_{\text{join}} \quad (1)$$

where θ_{join} is a predefined similarity threshold.

Union Discovery Problem. Given a query table T_q , the goal is to find a set of tables $C_{\text{union}} \subseteq \mathcal{D}$ where each table $T_u \in C_{\text{union}}$ pertains to the same class of entities as T_q . This requires assessing the holistic semantic similarity between tables, even if their schemas differ. This can be expressed as finding tables T_u such that:

$$\text{semantic_similarity}(T_q, T_u) > \theta_{\text{union}} \quad (2)$$

where θ_{union} is a predefined similarity threshold.

3.2 Multi-Agent Architecture

To address the dual challenges defined above, we propose a novel framework centered on multi-agent collaboration. As illustrated in Figure 2, the architecture is founded on the principles of separation of concerns, flexibility, and scalability. It comprises six specialized

agents orchestrated by a LangGraph-based workflow engine, which utilize a shared, three-layer filtering and verification toolset. This modularity allows for focused optimization and adaptive strategy execution.

The roles of the six agents are as follows: **ConfiguratorAgent:** This agent performs system-level configuration. Based on corpus statistics (e.g., table count) and available resources, it determines optimal parameters such as the degree of parallelization and caching strategies. **PlannerAgent:** Acting as the strategic orchestrator, this agent analyzes the query and task type (JOIN or UNION) to devise an optimal execution plan. For join discovery, it selects a bottom-up strategy focused on column compatibility; for union discovery, it chooses a top-down strategy prioritizing holistic table semantics. (Please refer to Appendix A.1 for the formalization of this decision logic and the complete operational workflow.) **AnalyzerAgent:** This agent conducts a comprehensive analysis of the query table T_q . It extracts a rich set of features—including structural properties (column types, cardinality), statistical distributions, and

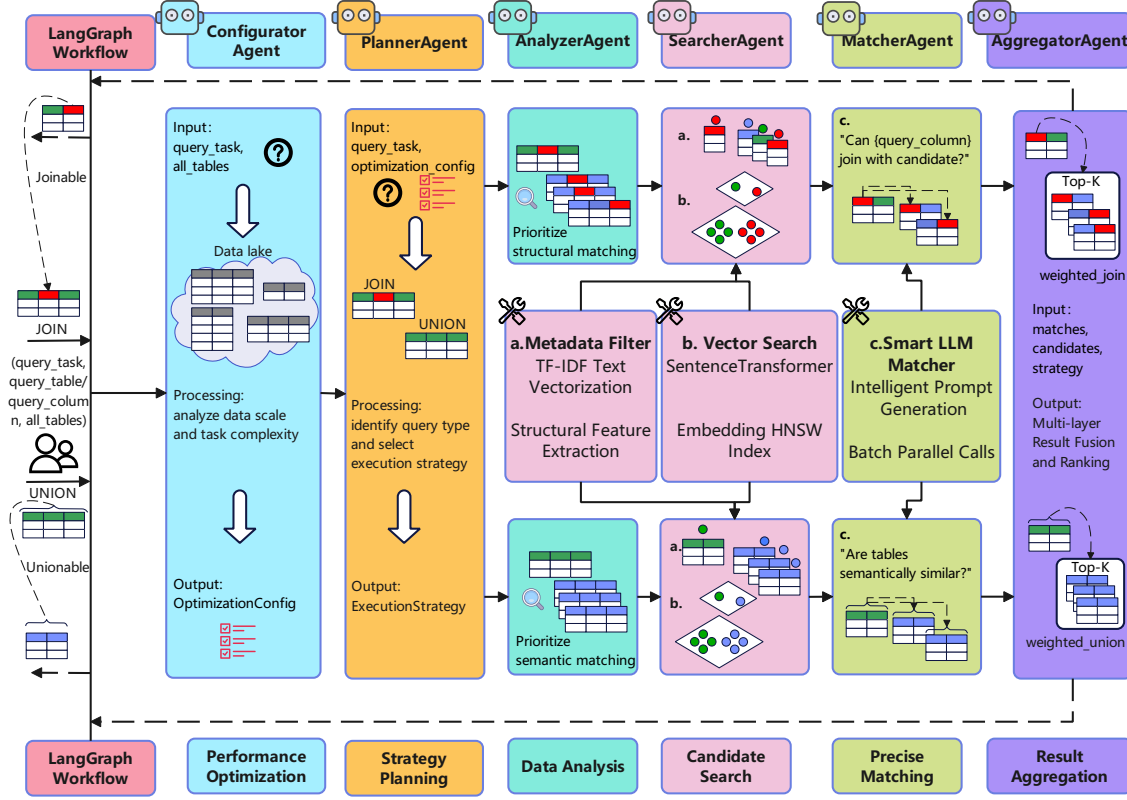


Figure 2: The proposed multi-agent framework architecture. The workflow, orchestrated by LangGraph, commences with the ConfiguratorAgent for system configuration. Subsequently, the PlannerAgent and AnalyzerAgent collaborate to devise a task-specific execution strategy and extract rich query features. The SearcherAgent and MatcherAgent then leverage three core modules—(a) Metadata Filter, (b) Vector Search, and (c) Smart LLM Matcher—to systematically identify and verify candidate tables. Finally, the AggregatorAgent synthesizes multi-layer evidence into a ranked output. The workflow trajectory adaptively pivots based on the specific integration intent (JOIN or UNION).

semantic embeddings—which are used by other agents in subsequent stages. **SearcherAgent:** This agent is the core component for candidate retrieval. It executes the search plan by invoking the Three-Layer Cascaded Filtering (TLCF) algorithm. It manages the flow of candidates from the fast metadata filter to the more precise semantic search layer. **MatcherAgent:** This agent is responsible for high-precision verification. It receives the small set of promising candidates from the SearcherAgent and uses an LLM to perform nuanced semantic validation. It employs task-specific prompts and intelligent batching to optimize for both accuracy and cost. **AggregatorAgent:** The final agent in the workflow, it synthesizes evidence from all previous stages. It combines the scores from each layer of the TLCF algorithm using task-specific weights to produce a final, ranked list of tables with associated confidence scores.

3.3 Three-Layer Cascaded Filtering (TLCF) Algorithm

The TLCF algorithm is the primary toolset shared by the agents. It operates as a progressive filtering pipeline composed of three distinct layers (L1, L2, L3), with each layer drastically reducing the candidate set while increasing matching precision. The high-level process is outlined in Algorithm 1.

Layer 1: Metadata Filter (L1) This initial layer, corresponding to component (a) in Figure 2, performs rapid, coarse-grained filtering. For a query table Q and a candidate table $t \in \mathcal{T}$, the L1 similarity score $S_1(Q, t)$ is a weighted combination of textual and structural features:

$$S_1(Q, t) = \alpha \cdot S_{\text{text}}(Q, t) + (1 - \alpha) \cdot S_{\text{struct}}(Q, t) \quad (3)$$

where α is a weighting coefficient. S_{text} is the cosine similarity of pre-computed TF-IDF vectors, and S_{struct} combines features such as the Jaccard similarity of column data types. Tables with a score

Algorithm 1 Three-Layer Cascaded Filtering (TLCF)**Require:** Query table Q , Table repository $\mathcal{T}$ (pre-indexed)**Ensure:** A ranked list of matched tables M **Layer 1 (L1): Metadata Filtering**

```

1:  $C_1 \leftarrow \emptyset$ 
2: for each table  $t \in \mathcal{T}$  do
3:    $s_1 \leftarrow \text{ComputeMetadataSimilarity}(Q, t)$ 
4:   if  $s_1 > \theta_1$  then
5:      $C_1 \leftarrow C_1 \cup \{(t, s_1)\}$ 
6:  $C_1 \leftarrow \text{TopK}(C_1, k_1)$ 

```

Layer 2 (L2): Vector Search

```

7:  $C_2 \leftarrow \emptyset$ 
8:  $\vec{e}_Q \leftarrow \text{Embed}(Q)$ 
9:  $\vec{N} \leftarrow \text{HnswSearch}(\mathcal{T}, \vec{e}_Q, k_2)$ 
10: for each  $(t, \text{dist}) \in \vec{N}$  where  $t \in \text{tables}(C_1)$  do
11:    $s_2 \leftarrow 1 - \text{dist}$ 
12:   if  $s_2 > \theta_2$  then
13:      $C_2 \leftarrow C_2 \cup \{(t, s_2)\}$ 

```

Layer 3 (L3): LLM Verification

```

14:  $C_3 \leftarrow \emptyset$ 
15: for each batch  $B \in \text{Partition}(\text{tables}(C_2))$  do
16:    $\text{results} \leftarrow \text{LLMVerify}(Q, B, \tau)$ 
17:    $C_3 \leftarrow C_3 \cup \text{results}$ 

```

Final Score Aggregation

```

18:  $M \leftarrow \text{AggregateScores}(C_1, C_2, C_3)$ 
19: return  $\text{SortByScore}(M)$ 

```

$S_1(Q, t)$ exceeding a threshold θ_1 form the initial candidate set C_1 , which is then pruned to the top k_1 .

Layer 2: Vector Search (L2) This layer, shown as component (b) in Figure 2, refines C_1 based on deep semantic similarity. A pre-trained sentence-transformer is used to generate an embedding $\vec{e}$ for each table. The L2 similarity score $S_2(Q, t)$ is the cosine similarity between the embeddings of the query table, $\vec{e}_Q$, and a candidate table, $\vec{e}_t$:

$$S_2(Q, t) = \cos(\vec{e}_Q, \vec{e}_t) \quad (4)$$

Using an HNSW index pre-built on $\mathcal{T}$, the top k_2 nearest neighbors are retrieved along with their cosine distances (dist). Since S_2 represents similarity, we calculate it as $1 - \text{dist}$. Only those neighbors that are also present in C_1 and have a score above a threshold θ_2 are retained to form the set C_2 .

Layer 3: Smart LLM Matcher (L3) The final layer, component (c) in Figure 2, aims for maximum precision. The MatcherAgent sends candidates from C_2 to an LLM. The LLM is provided with a task-specific prompt, $\tau \in \{\text{JOIN}, \text{UNION}\}$, and returns a high-confidence verification score $S_3(Q, t)$:

$$S_3(Q, t) = \text{LLM}(Q, t, \tau) \quad (5)$$

Only tables with a score S_3 exceeding a threshold θ_3 form the final set of matches C_3 .

Final Score Aggregation The AggregatorAgent synthesizes evidence from all layers. The final score $S_{\text{final}}(t)$ for each table is a weighted sum of its normalized scores ($\hat{S}_i$) from each layer, using

task-specific weights (w_i):

$$S_{\text{final}}(t) = \sum_{i=1}^3 w_i \cdot \hat{S}_i(t), \quad \text{where } \sum_{i=1}^3 w_i = 1 \quad (6)$$

3.4 Agent Orchestration with LangGraph

The cooperative interactions between agents are managed by a stateful graph implemented within the LangGraph framework. The execution flow proceeds sequentially, commencing with the ConfiguratorAgent and concluding with the AggregatorAgent, where the output of each agent is appended to a shared WorkflowState object. This state acts as a centralized context, maintaining intermediate results and shared, pre-computed resources (e.g., the L1 TF-IDF matrix and the L2 HNSW index). This design ensures that computationally intensive operations are performed only once during initialization, allowing each subsequent query to efficiently access these resources for scalable and high-performance execution.

4 Experimental Evaluation**4.1 Experimental Setup**

4.1.1 Datasets. To evaluate our framework's performance across diverse scenarios, we employ three real-world datasets, each representing a different facet of the Web data ecosystem. Their statistics are summarized in Table 1.

WebTable[7]-noise: Derived from Common Crawl, this corpus serves as our primary benchmark for evaluating discovery performance on noisy Web data. To simulate realistic heterogeneity that bypasses simple exact string matching, the original corpus was augmented with Valentine-style [20] noise. Specifically, *schema noise* (e.g., prefixes like "Team" $\rightarrow$ "col_TEAM", abbreviations, and synonyms like "avg" $\rightarrow$ "mean") was injected, and *instance noise* (e.g., typos and $\pm 5\%$ numeric perturbations) was applied to approximately 30% of candidate columns. This configuration necessitates robust semantic understanding to effectively overcome superficial lexical variations and accurately identify related tables.

OpenData[7]: This dataset is composed of higher-quality tables sourced from various government open data portals. These tables are well-structured but are published independently without explicit links, posing a challenge of discovering relationships across different domains.

NLCTables[5] This dataset contains tables that are often accompanied by natural language descriptions. It is designed to test a system's ability to perform fine-grained semantic understanding, making it suitable for evaluating the more nuanced aspects of our matching capabilities.

All datasets utilize randomized identifiers for tables to ensure that discovery methods rely on genuine structural and semantic signals rather than superficial name matching.

4.1.2 Implementation Details. Implemented in Python 3.10 using LangGraph, our framework utilizes scikit-learn (TF-IDF) for the L1 stage, *all-MiniLM-L6-v2* embeddings with FAISS (HNSW) for L2 retrieval, and Gemini 1.5 Flash (temperature 0.1) for L3 verification. Key parameters were optimized on a validation set. All experiments were conducted on a high-performance server equipped with an Intel Xeon Platinum CPU, 1TB of RAM, and an NVIDIA

Table 1: Dataset Statistics Summary

Dataset	Task	Queries	Tables	GT	GT/Query	Table Rows		Table Columns
						Mean	Range	Mean (Range)
WebTable-noise	JOIN	230	1,534	2,133	9.3	77.09	5–477	9.51 (3–24)
	UNION	823	5,487	14,850	18.0	61.08	5–640	10.24 (3–24)
OpenData	JOIN	75	500	1,077	14.4	982.72	49–1,000	20.46 (3–195)
	UNION	464	3,095	12,343	26.6	969.37	5–1,000	19.85 (3–329)
NLCTables	JOIN	91	4,872	211	2.3	3,543.9	20–58,900	9.40 (3–28)
	UNION	249	7,567	815	3.3	104.5	10–1,000	13.50 (3–25)
WebTable-noise Total		1,053	7,021	16,983	16.1	68.36	5–640	9.96 (3–24)
OpenData Total		539	3,595	13,420	24.9	975.84	5–1,000	20.48 (3–329)
NLCTables Total		340	12,439	1,026	3.0	1,324.6	10–58,900	9.65 (3–28)

A100 GPU. The complete source code and experimental data are publicly available for reproducibility¹. For baseline details, please refer to Appendix A.2.

4.1.3 Evaluation Metrics. We employ comprehensive evaluation metrics to assess retrieval effectiveness: Precision@k, Recall@k, and NDCG@k (where k=1, 5, 10). These metrics measure the quality, completeness, and ranking of the search results at different cut-off points. **Precision@k** is the fraction of relevant tables among the top-k retrieved results, indicating the accuracy of the search. **Recall@k** is the fraction of all existing relevant tables that are found within the top-k results, measuring the search’s coverage. **NDCG@k (Normalized Discounted Cumulative Gain)** evaluates ranking quality by assigning higher weights to relevant items at top-tier positions, contrasting with top-k Precision which treats all hits uniformly regardless of their rank. The metric is derived from the Discounted Cumulative Gain ($DCG@k$), defined as follows:

$$DCG@k = \sum_{i=1}^k \frac{rel_i}{\log_2(i+1)} \quad (7)$$

where rel_i represents the relevance score of the result at rank i . To account for varying numbers of relevant items across queries, $DCG@k$ is normalized by the Ideal Discounted Cumulative Gain ($IDCG@k$), which is the maximum possible DCG achieved by a perfect ranking of all available relevant items:

$$NDCG@k = \frac{DCG@k}{IDCG@k} \quad (8)$$

This metric quantifies the utility of retrieving relevant results within the top ranks, where a value of 1.0 denotes an ideal ranking state.

4.2 Experimental Results

We conducted a comprehensive set of experiments to evaluate our proposed multi-agent framework. Our evaluation is designed to answer three key questions: (1) How does our framework’s overall efficacy compare against state-of-the-art baseline methods? (2) What is the contribution of each agent to the system’s performance? (3) What is the cost-performance trade-off of using different Large Language Models?

¹<https://github.com/Zaiwen/DLMulti>

4.2.1 Overall Efficacy Comparison. We first compare the end-to-end performance of our multi-agent framework against a suite of established baseline methods on both JOIN and UNION discovery tasks. The results, presented in Figure 3, demonstrate the effectiveness of our approach across three diverse datasets.

For the JOIN task, our method consistently outperforms baselines, most notably on the challenging WebTable-noise dataset where it exhibits significant improvements in Recall and NDCG relative to established methods such as Josie and DeepJoin. This performance underscores the framework’s robustness against both lexical and semantic noise. On the OpenData corpus, for instance, we achieve a Precision@1 of 0.533 alongside superior recall (0.639 @10). These results confirm that our cascaded filtering effectively balances precision and coverage; in contrast to DeepJoin, which successfully captures semantics but tends to incorporate irrelevant candidates, our L3 LLM verification layer effectively eliminates false positives.

For the UNION task, our framework demonstrates superior performance, particularly excelling against strong baselines such as Santos and STARMIE in terms of NDCG across all datasets. On OpenData, for example, we achieve an NDCG@10 of 0.688, surpassing all competitors. This success is primarily credited to the synergistic TLCF design: the L2 vector search captures broad semantic relevance, while the L3 LLM verification stage provides the fine-grained contextual reasoning necessary to ensure optimal ranking quality.

Efficiency Analysis. Table 2 details the offline (index construction) and online (per-query) execution times. Our method achieves highly competitive offline times, as it relies on efficient indexing techniques like TF-IDF and HNSW. While our online query time is higher than some lightweight baselines (e.g., LSH), this is an expected trade-off for the significant accuracy gains provided by the L3 LLM verification stage. Crucially, our cascaded filtering ensures that the computationally expensive LLM is invoked only on a small, highly relevant set of candidates, which ensures the practicality of the approach.

4.2.2 Ablation Study: The Impact of Multi-Agent Collaboration. We validate our architecture through an ablation study, systematically removing individual agents and evaluating the system against a single-agent ReAct [31] baseline (see Table 3). The full system

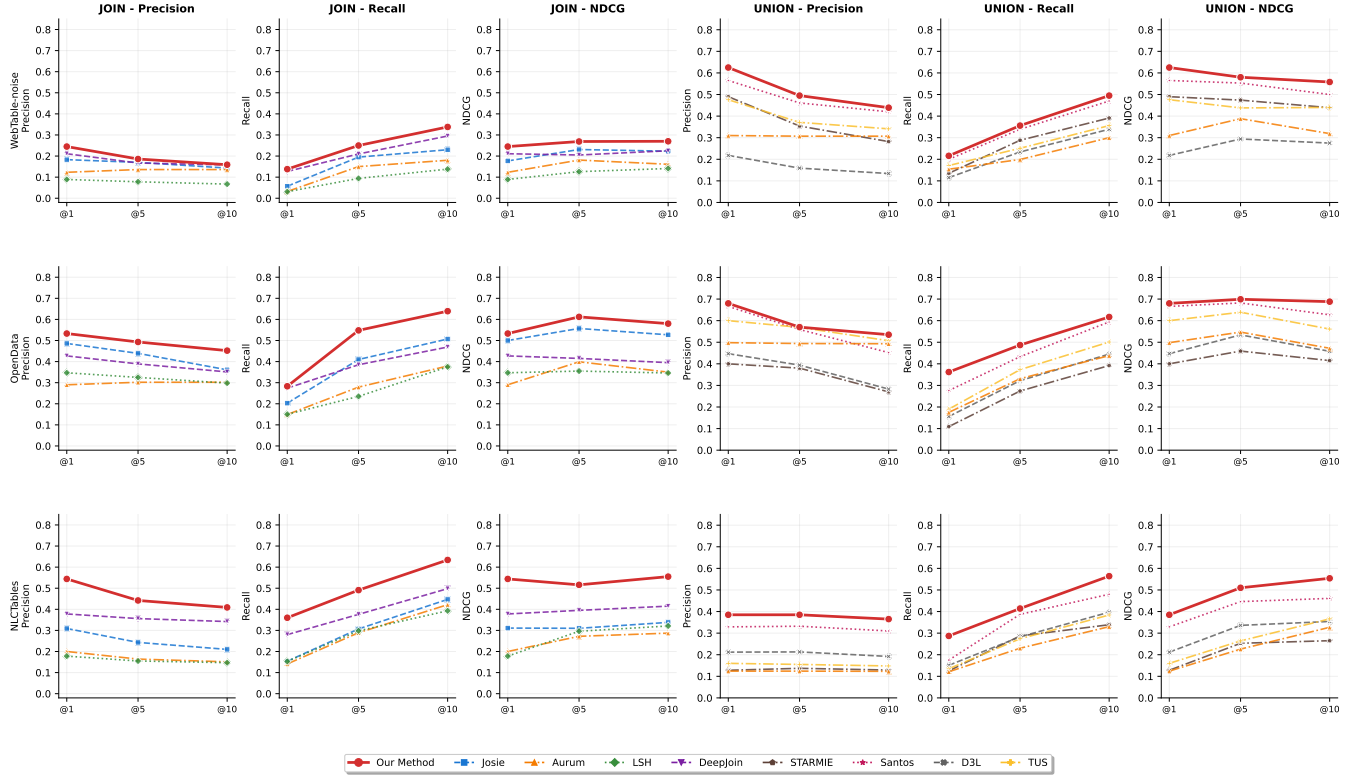


Figure 3: Comprehensive performance comparison of our method against baselines on Precision@k, Recall@k, and NDCG@k for JOIN (left) and UNION (right) tasks across the WebTable-noise, OpenData, and NLCtables datasets.

consistently outperforms all ablated versions. Empirical results demonstrate that removing any specialized agent degrades performance; notably, omitting the *SearcherAgent* drastically increases the computational load on subsequent stages, while removing the *MatcherAgent* eliminates L3 verification, thereby significantly reducing precision. Most significantly, our framework outperforms the ReAct baseline, achieving a substantial improvement in JOIN Precision@1 of approximately 70% (from 0.312 to 0.533) while simultaneously reducing execution latency. This validates our design of employing specialized, collaborative agents over a monolithic reasoning process.

4.2.3 Cost-Performance Analysis of LLMs. We evaluated several models on the WebTable UNION task to assess the trade-off between cost and accuracy (see Table 4). Results indicate that lightweight models, such as Gemini-1.5-Flash, offer the optimal balance: delivering robust performance (P@1 of 0.625) at a minimal cost (~\$2.15 for over 21,000 calls). While GPT-4o offers marginal accuracy gains (~7.5%), it incurs over 30 times the operational cost, rendering it impractical for large-scale discovery at production scales. These findings confirm that leveraging efficient, high-performance LLMs is essential for the practical viability of the framework.

5 Conclusion and Future Work

In this paper, we introduced a unified, time-efficient multi-agent framework designed to resolve the long-standing efficiency-accuracy

trade-off in Web table discovery. Extensive experiments demonstrate that our approach consistently outperforms state-of-the-art methods across diverse and challenging datasets. The effectiveness of our framework stems from specialized agents orchestrating the Three-Layer Cascaded Filtering (TLCF) algorithm, which adaptively handles join and union tasks while systematically pruning the search space to ensure computational efficiency. These findings validate multi-agent architectures as a robust and scalable paradigm for large-scale data discovery.

Despite these promising results, challenges regarding Large Language Model (LLM) latency and operational costs remain. Future work will investigate model distillation to mitigate verification costs and extend the framework’s capabilities to identify more complex hierarchical relationships between tables. Finally, we aim to facilitate dynamic strategy adaptation via multi-agent reinforcement learning, moving towards a truly autonomous and self-optimizing data discovery system.

Acknowledgments

The authors from Huazhong Agricultural University were supported by the National Key Research and Development Program of China (Grant No. 2023YFF1000100), the Hubei Key Research and Development Program of China (Grants 2024BBB055, 2024BAA008), and the Fundamental Research Funds for the Central Universities (Grant 2662025XXPY005).

Table 2: Offline and Online Time Comparison of Baseline Methods (in seconds)

Task	Method	WebTable-noise		OpenData		NLCTables	
		Offline	Online (avg.)	Offline	Online (avg.)	Offline	Online (avg.)
JOIN	Josie	52.34	28.67	18.92	14.33	64.78	39.45
	Aurum	207.89	89.24	53.67	29.81	256.43	132.76
	LSH	29.45	18.93	10.67	9.24	41.28	30.89
	DeepJoin	1456.72	142.35	218.94	41.67	2087.33	174.82
	Our Method	21.58	161.56	9.47	182.72	56.23	93.61
UNION	STARMIE	156.78	1456.23	104.56	743.89	187.45	1398.67
	Santos	987.45	523.78	534.67	334.92	1467.89	262.45
	D3L	412.67	218.94	247.83	143.56	598.23	189.67
	TUS	534.89	378.45	335.67	238.92	776.34	334.78
	Aurum	789.23	298.67	449.78	187.45	967.45	219.83
	Our Method	72.45	1159.70	43.68	718.17	92.56	362.34

Table 3: Multi-Agent Ablation Study on the OpenData Dataset

Task	Configuration	Precision@k			Recall@k			NDCG@k			Time(s)
		P@1	P@5	P@10	R@1	R@5	R@10	N@1	N@5	N@10	
JOIN	Single-Agent ReAct	0.312	0.283	0.267	0.178	0.312	0.445	0.312	0.389	0.421	245.6
	w/o ConfiguratorAgent	0.498	0.456	0.423	0.258	0.501	0.598	0.498	0.572	0.548	172.7
	w/o PlannerAgent	0.445	0.398	0.371	0.234	0.445	0.556	0.445	0.509	0.493	167.7
	w/o AnalyzerAgent	0.509	0.467	0.431	0.267	0.523	0.612	0.509	0.585	0.562	162.7
	w/o SearcherAgent	0.387	0.345	0.321	0.203	0.398	0.502	0.387	0.456	0.442	152.7
	w/o MatcherAgent	0.517	0.481	0.431	0.263	0.527	0.619	0.517	0.595	0.563	82.7
	w/o AggregatorAgent	0.521	0.475	0.438	0.271	0.531	0.625	0.521	0.591	0.568	174.7
	Full Multi-Agent	0.533	0.493	0.452	0.283	0.548	0.639	0.533	0.612	0.580	182.72
UNION	Single-Agent ReAct	0.487	0.423	0.378	0.234	0.334	0.467	0.487	0.521	0.545	923.4
	w/o ConfiguratorAgent	0.651	0.542	0.498	0.321	0.445	0.578	0.651	0.657	0.651	678.2
	w/o PlannerAgent	0.612	0.515	0.467	0.298	0.412	0.541	0.612	0.628	0.623	658.2
	w/o AnalyzerAgent	0.645	0.538	0.498	0.315	0.445	0.578	0.645	0.651	0.642	638.2
	w/o SearcherAgent	0.598	0.512	0.467	0.289	0.412	0.534	0.598	0.612	0.608	598.2
	w/o MatcherAgent	0.660	0.560	0.515	0.345	0.465	0.599	0.660	0.676	0.672	318.2
	w/o AggregatorAgent	0.668	0.561	0.521	0.351	0.472	0.605	0.668	0.682	0.675	700.2
	Full Multi-Agent	0.680	0.570	0.535	0.362	0.487	0.617	0.680	0.699	0.688	718.17

Table 4: LLM Model Cost-Performance Analysis for the Multi-Agent System on the WebTable UNION Task (823 queries, 21,398 total LLM calls)

Model	Provider	Cost per 1M Tokens (In/Out)	Total Tokens (M)	Total Cost (\$)	P@1 / R@1
<i>Lightweight Models</i>					
Gemini-1.5-Flash	Google	\$0.075 / \$0.30	21.4	2.15	0.625 / 0.216
GPT-4o-mini	OpenAI	\$0.15 / \$0.60	21.8	4.40	0.618 / 0.212
Claude-3.5-Haiku	Anthropic	\$0.80 / \$4.00	20.9	23.44	0.605 / 0.205
<i>Advanced Models (For Comparison)</i>					
Gemini-2.0-Flash	Google	\$0.10 / \$0.40	21.6	2.88	0.635 / 0.223
Gemini-2.5-Pro	Google	\$1.25 / \$10.00	22.3	49.75	0.658 / 0.238
GPT-4o	OpenAI	\$2.50 / \$10.00	22.5	75.00	0.672 / 0.245

References

- [1] Alex Bogatu, Alvaro A. A. Fernandes, Norman W. Paton, and Nikolaos Konstantinou. 2020. Dataset Discovery in Data Lakes. In *Proceedings of the 36th IEEE International Conference on Data Engineering (ICDE)*. 709–720. doi:10.1109/ICDE48307.2020.00067
- [2] Sara Bonfitto, E. Casiraghi, and M. Mesiti. 2021. Table understanding approaches for extracting knowledge from heterogeneous tables. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 11 (2021). doi:10.1002/widm.1407
- [3] F. Chirigati, Rémi Rampin, Aécio Santos, Aline Bessa, and J. Freire. 2021. Auctus: A Dataset Search Engine for Data Discovery and Augmentation. *Proc. VLDB*

- Endow.* 14 (2021), 2791–2794.
- [4] M. Cremaschi, F. D. Paoli, Anisa Rula, and Blerina Spahiu. 2020. A fully automated approach to a complete Semantic Table Interpretation. *Future Gener. Comput. Syst.* 112 (2020), 478–500. doi:10.1016/j.future.2020.05.019
 - [5] Lingxi Cui, Huan Li, Ke Chen, Lidan Shou, and Gang Chen. 2025. nlcTables: A Dataset for Marrying Natural Language Conditions with Table Discovery. *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (2025). doi:10.1145/3726302.3730296
 - [6] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. 2020. TURL. *ACM SIGMOD Record* 51 (2020), 33–40. doi:10.1145/3542700.3542709
 - [7] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *Proc. VLDB Endow.* 17 (2024), 1925–1938. doi:10.14778/3659437.3659448
 - [8] Yuyang Dong, Kunihiro Takeoka, Chuan Xiao, and Masafumi Oyamada. 2021. Efficient Joinable Table Discovery in Data Lakes: A High-Dimensional Similarity-Based Approach. In *Proceedings of the 37th IEEE International Conference on Data Engineering (ICDE)*. 456–467. doi:10.1109/ICDE51399.2021.00046
 - [9] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and M. Oyamada. 2022. DeepJoin: Joinable Table Discovery with Pre-trained Language Models. *ArXiv abs/2212.07588* (2022). doi:10.14778/3603581.3603587
 - [10] Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *Companion of the 2023 ACM SIGMOD International Conference on Management of Data*. 69–75. doi:10.1145/3555041.3589409
 - [11] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2023. Semantic-aware Dataset Discovery from Data Lakes with Contextualized Column-based Representation Learning. *Proceedings of the VLDB Endowment* 16, 7 (2023), 1726–1739. doi:10.14778/3587136.3587146
 - [12] R. Fernandez, Ziawasch Abedjan, F. Koko, Gina Yuan, S. Madden, and M. Stonebraker. 2018. AURUM: A Data Discovery System. *2018 IEEE 34th International Conference on Data Engineering (ICDE)* (2018), 1001–1012. doi:10.1109/icde.2018.00094
 - [13] M. Ferrag, Norbert Tihanyi, and M. Debbah. 2025. From LLM Reasoning to Autonomous AI Agents: A Comprehensive Review. *ArXiv abs/2504.19678* (2025). doi:10.48550/arXiv.2504.19678
 - [14] A. Helal, M. Helali, Khaled Ammar, and Essam Mansour. 2021. A Demonstration of KGLac: A Data Discovery and Enrichment Platform for Data Science. *Proc. VLDB Endow.* 14 (2021), 2675–2678. doi:10.14778/3476311.3476317
 - [15] Aidan Hogan, E. Blomqvist, Michael Cochez, C. d'Amato, Gerard de Melo, C. Gutierrez, J. E. L. Gayo, S. Kirrane, S. Neumaier, A. Polleres, Roberto Navigli, A. Ngomo, S. M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. 2020. Knowledge Graphs. *ACM Computing Surveys (CSUR)* 54 (2020), 1–37. doi:10.1145/3447772
 - [16] Anna V. Kalyuzhnaya, Sergey Mityagin, Elizaveta Lutsenko, Andrey Getmanov, Yaroslav Aksentkin, Kamil Fatkhiev, Kirill Fedorin, Nikolay O. Nikitin, Natalia Chichkova, Vladimir Vorona, and Alexander Boukhanovsky. 2025. LLM Agents for Smart City Management: Enhancing Decision Support Through Multi-Agent AI Systems. *Smart Cities* (2025). doi:10.3390/smartcities8010019
 - [17] M. Kasem, Abdelrahman Abdallah, Alexander Berendeyev, Ebrahim Elkady, Mahmoud Abdalla, Mohamed Mahmoud, Mohamed Hamada, D. Nursetov, and I. Taj-Eddin. 2022. Deep Learning for Table Detection and Structure Recognition: A Survey. *Comput. Surveys* 56 (2022), 1–41. doi:10.1145/3657281
 - [18] Moe Kayali, A. Lykov, Ilias Fountalis, N. Vasiloglou, Dan Olteanu, and Dan Suciu. 2023. CHORUS: Foundation Models for Unified Data Discovery and Exploration. *ArXiv abs/2306.09610* (2023). doi:10.48550/arXiv.2306.09610
 - [19] Aamod Khatiwada, Grace Fan, Roe Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationshipbased Semantic Table Union Search. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–25. doi:10.1145/3588689
 - [20] Christos Koutras, George Siachamis, Andra Ionescu, Kyriakos Psarakis, Jerry Brons, Marios Fragkoulis, Christoph Lofi, Angela Bonifati, and Asterios Katsifodimos. 2021. Valentine: Evaluating Matching Techniques for Dataset Discovery. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, IEEE, 468–479. doi:10.1109/ICDE51399.2021.00047
 - [21] LangChain Team. 2025. *LangGraph: A library for building stateful, multi-agent applications*. <https://github.com/langchain-ai/langgraph>
 - [22] Jhomara Luzuriaga, Emir Muñoz, Henry Rosales-Méndez, and Aidan Hogan. 2023. Merging Web Tables for Relation Extraction With Knowledge Graphs. *IEEE Transactions on Knowledge and Data Engineering* 35 (2023), 1803–1816. doi:10.1109/TKDE.2021.3101479
 - [23] Fatemeh Nargesian, Erkang Zhu, Ken Q. Pu, and Renée J. Miller. 2018. Table union search on open data. *Proceedings of the VLDB Endowment* 11, 7 (2018), 813–825. doi:10.14778/3192965.3192973
 - [24] Tuan-Phong Nguyen, S. Razniewski, Julien Romero, and G. Weikum. 2021. Refined Commonsense Knowledge From Large-Scale Web Contents. *IEEE Transactions on Knowledge and Data Engineering* 35 (2021), 8431–8447. doi:10.1109/TKDE.2022.3206505
 - [25] Vy Nguyen, Ziqi Xu, Jeffrey Chan, Estrid He, Feng Xia, and Xiuzhen Zhang. 2025. Hallucinate Less by Thinking More: Aspect-Based Causal Abstention for Large Language Models. *arXiv preprint arXiv:2511.17170* (2025).
 - [26] Bowen Qin, Binyuan Hui, Lihan Wang, Min Yang, Binhua Li, Fei Huang, Luo Si, Qingshan Jiang, and Yongbin Li. 2023. Schema dependency-enhanced curriculum pre-training for table semantic parsing. *Knowl. Based Syst.* 262 (2023), 110264. doi:10.1016/j.knosys.2023.110264
 - [27] Jing Ren, Wenhao Zhou, Bowen Li, Mujie Liu, Nguyen Linh Dan Le, Jiade Cen, Liping Chen, Ziqi Xu, Xiwei Xu, and Xiaodong Li. 2025. Causal Prompting for Implicit Sentiment Analysis with Large Language Models. *CoRR abs/2507.00389* (2025).
 - [28] G. Weikum. 2021. Knowledge Graphs 2021: A Data Odyssey. *Proc. VLDB Endow.* 14 (2021), 3233–3238. doi:10.14778/3476311.3476393
 - [29] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Qin Liu, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huan, and Tao Gui. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. *ArXiv abs/2309.07864* (2023). doi:10.48550/arXiv.2309.07864
 - [30] Senqi Yang, Dongyu Zhang, Jing Ren, Ziqi Xu, Xiuzhen Zhang, Yiliao Song, Hongfei Lin, and Feng Xia. 2025. Cultural Bias Matters: A Cross-Cultural Benchmark Dataset and Sentiment-Enriched Model for Understanding Multimodal Metaphors. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL. 26301–26317.
 - [31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *ArXiv abs/2210.03629* (2022).
 - [32] Haochen Zhang, Yuyang Dong, Chuan Xiao, and Masafumi Oyamada. 2023. Jellyfish: A large language model for data preprocessing. *arXiv preprint arXiv:2312.01678* (2023).
 - [33] Shuo Zhang and K. Balog. 2020. Web Table Extraction, Retrieval, and Augmentation. *ACM Transactions on Intelligent Systems and Technology (TIST)* 11 (2020), 1–35. doi:10.1145/3372117
 - [34] Bo Zhao, Yinghao Zhang, Ziqi Xu, Yongli Ren, Xiuzhen Zhang, Renqiang Luo, Zaiwen Feng, and Feng Xia. 2025. Unbiased Reasoning for Knowledge-Intensive Tasks in Large Language Models via Conditional Front-Door Adjustment. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM*. 4315–4325.
 - [35] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J. Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. ACM, New York, NY, USA, 847–864. doi:10.1145/3299869.3300065
 - [36] Erkang Zhu, Fatemeh Nargesian, K. Pu, and Renée J. Miller. 2016. LSH Ensemble: Internet-Scale Domain Search. *ArXiv abs/1603.07410* (2016). doi:10.14778/2994509.2994534

A Appendix

A.1 Framework Operational Workflow and Planner Logic

The execution of our multi-agent framework follows a systematic, cooperative workflow designed to simultaneously optimize efficiency and accuracy. The process initiates with the *ConfiguratorAgent*, which performs a system-level assessment of the data scale and available computational resources to determine operational parameters such as parallelization degrees and caching strategies. Following this initialization, core decision-making responsibility shifts to the *PlannerAgent*. To provide rigorous clarity regarding its decision logic, we formalize the planner as a deterministic mapping function $f_{\text{plan}} : (Q, T) \rightarrow \Phi$, where Q represents the query context and $T \in \{\text{JOIN}, \text{UNION}\}$ denotes the task type. The generated output configuration Φ encompasses the layer-wise aggregation weights $W = \{w_{L1}, w_{L2}, w_{L3}\}$, a task-specific prompt template P , and a set of filtering thresholds $\Theta = \{\theta_1, \theta_2, \theta_3\}$.

The strategy instantiation within the *PlannerAgent* is adaptively adjusted based on the integration intent. Specifically, for JOIN tasks, the planner activates a Bottom-Up strategy by assigning a specific weight vector W_{join} tailored to the structural nature of the task.

Conversely, for UNION tasks, the planner adopts a Top-Down strategy with a distinct weight vector W_{union} , which is configured to prioritize the semantic relatedness essential for unionability. This mechanism ensures that the final ranking logic of the *AggregatorAgent* is dynamically aligned with the specific requirements of the task at hand.

Complementing this strategic formulation, the *AnalyzerAgent* conducts a deep structural and semantic analysis of the query table. It extracts a comprehensive set of features, including column data types and statistical distributions, to support the structural compatibility checks in L1. Furthermore, it prepares the semantic representation of the query to enable the retrieval process of the *SearcherAgent*. In the subsequent L2 layer, the *SearcherAgent* leverages these query features to perform efficient retrieval against the pre-computed and cached embeddings of the data lake via the HNSW index. With the strategy configured and candidates retrieved, the *MatcherAgent* performs high-precision verification via the TLCF algorithm. Finally, the *AggregatorAgent* synthesizes the multi-layer signals based on the planner's weight configuration to produce the final ranked results.

A.2 Baseline Implementation Details

To ensure a fair and comprehensive evaluation, we benchmarked our framework against three categories of state-of-the-art methods, utilizing their official implementations or high-fidelity reproductions wherever feasible.

Schema and Keyword-based Methods. We evaluate our framework against Josie [35], Aurum [12], and LSH Ensemble [36]. Josie is a specialized method for exact set-overlap search designed for join discovery; it utilizes an inverted index over cell values to efficiently identify joinable tables and employs a cost model to prune

candidates. Aurum is a data discovery system that constructs a comprehensive enterprise knowledge graph, connecting tables through relationships derived from schema-level similarity and content overlap. Additionally, LSH Ensemble is a pioneering technique for set-containment search on data lakes. It introduces a partitioning scheme over MinHash signatures to accurately estimate set containment, enabling the efficient discovery of joinable tables where the query values are a subset of the candidate column.

Embedding-based Methods. In this category, we evaluate against DeepJoin [9] and D3L [1]. DeepJoin is a semantic join-discovery method that fine-tunes pre-trained language models to compute the semantic similarity between column pairs. By converting columns into dense vectors and indexing them using Hierarchical Navigable Small World (HNSW) graphs, it remains effective for identifying joinable columns even when value overlap is low or fuzzy. D3L represents a representative distance-based framework that integrates diverse similarity signals, including attribute names, value distributions, data formats, and static word embeddings (e.g., fastText). It leverages Locality-Sensitive Hashing (LSH) forests for the efficient indexing of these multi-modal features, thereby enabling scalable similarity search.

System-based Methods. Finally, we compare against Starmie [11], Santos [19], and TUS [23], which are primarily designed for union search. Starmie is a scalable system that leverages contrastive learning to train column encoders, generating unified table representations by aggregating column vectors and utilizing an HNSW index for efficient retrieval. Santos improves union search accuracy by leveraging external knowledge bases to model the semantic relationships between columns, creating a semantic graph for each table to measure relatedness. TUS (Table Union Search) adopts a probabilistic approach that incorporates value overlap, ontology similarity, and natural language similarity to identify unionable tables within open data repositories.